NOVA IMS
Master in Data Science and Advanced Analytics

Predicting FIFA Player Overall Rating

An End-to-End MLOps Pipeline on the European Soccer Database

Group Members

Name	**Student No.**
Martim Líbano Monteiro	20250429
Simão José	20250410
Francisco Henriques	20250417
Pedro Carrasqueira	20250488
António Parafita	20250363

Machine Learning Operations (MLOps)
2025/2026

1 Problem, data and success metrics

We predict a football player's FIFA `overall_rating`, a bounded integer rating on the 0 to 100 scale, from in-game skill attributes. A scouting platform could use this to rank transfer targets automatically, and a game studio could use it to flag rating errors before a release ships.

The data is the European Soccer Database (Kaggle, Mathien), a SQLite database covering 2008 to 2016 across 11 European leagues. We work with the `Player_Attributes` table, about 184,000 per-player FIFA snapshots holding 33 skill ratings on a 0 to 100 scale plus `preferred_foot` and two work-rate fields, enriched with height, weight and birthday joined from the `Player` table. It is a good test bed because it is messy in realistic ways: mixed types, missing values, dirty categorical fields, a near-duplicate column that leaks the target, and a date column to work around. The non-null target values are integer ratings from 42 to 90, with 30 missing targets removed during cleaning. The committed CSV is a deterministic 6,000-row sample (seed 42) so the pipeline runs end to end in about two minutes; `scripts/build_dataset.py` regenerates the table at any size from the SQLite file.

On the committed sample the target is bounded and roughly symmetric (about 42 to 90, mean about 68), so no log transform is needed and we model the rating directly. We fixed the success criteria in Table 1 before modelling and report them on a held-out test set.

Metric	Role	Target (PoC)	Achieved
RMSE (pts)	Primary; penalises large errors	≤ 3.0	**1.45**
MAE (pts)	Typical absolute error	≤ 2.5	**1.05**
R^2	Share of rating variance explained	≥ 0.85	**0.954**
MAPE (%)	Relative error for stakeholders	≤ 5	**1.57**
Within 2 pts (%)	How often a prediction is close	≥ 70	**86.8**

Table 1: Success metrics, targets and results. The champion (HistGradientBoosting) meets every criterion on the held-out test set.

2 Project planning and pipeline architecture

We planned the project as a sequence of six weekly sprints, one per course topic, and built each sprint as a standalone Kedro pipeline rather than a notebook step: data quality first, then cleaning and the feature store, then modelling and MLflow tracking, then explainability, then serving and containers, then drift monitoring and tests. That choice means every sprint's output is also the production-runnable unit it would be in a real deployment, not a one-off script.

The project follows the Kedro layered data convention (`01_raw` through `08_reporting`). Figure 1 shows how the resulting eight pipelines connect: a straight line prepares the data, then two branches share the trained champion. `kedro run` executes the whole DAG in order, and `kedro run --pipeline <name>` runs one piece on its own. The second mode is the realistic production case once a model is live: `model_predict` scores new batches and `data_drifts` watches a fresh batch for distribution shift, and neither needs to retrain anything to do its job.

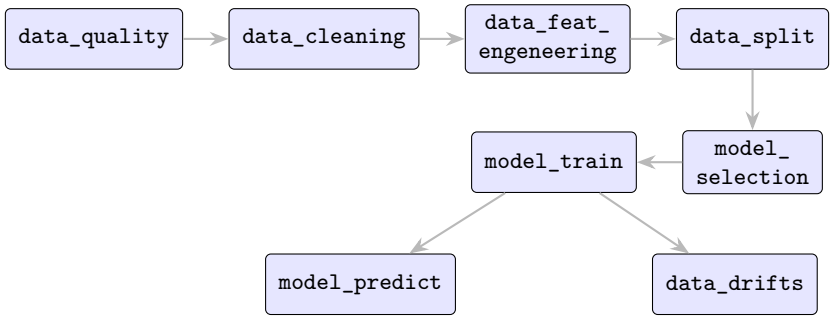


Figure 1: Pipeline DAG. The upper row prepares data, `model_selection` and `model_train` produce the champion, and `model_predict` and `data_drifts` consume it downstream.

A single `modeling.py` module holds the full preprocessing and estimator chain, so the model compared during cross-validation is the same object that is trained and served. The serving layer calls the same `preprocess_for_inference` routine as the offline pipeline, so the two paths can't drift apart. We'd tighten this further with a row-level test that checks both paths produce identical output on the same input.

3 Results and conclusions

This section follows the order we actually reasoned in: what we removed before training, how well the resulting model fits and how far we trust that fit, what the model relies on, and whether it notices when the data changes.

3.1 Data exploration and cleaning

The notebook `exploratory_data_analysis.ipynb` looks at the raw sample before any pipeline runs on it, and that exploration shaped four cleaning decisions. Figure 2 shows the two findings that mattered most: the shape of the target, and which raw attributes line up with it.

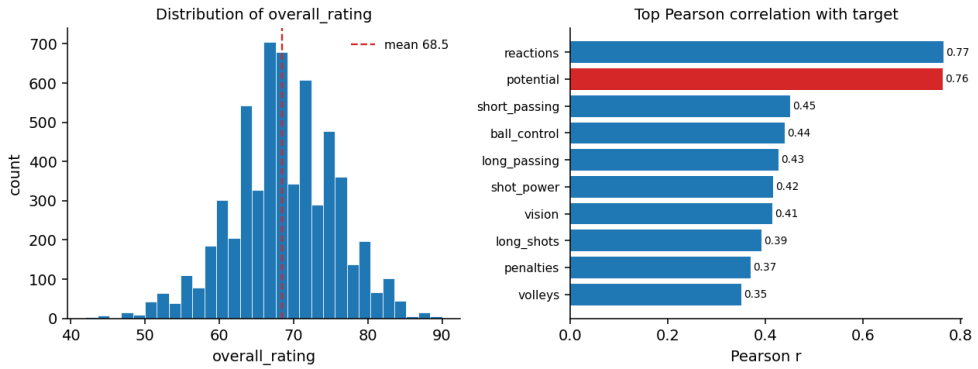


Figure 2: Exploratory analysis on the committed sample. Left: `overall_rating` is bounded and roughly symmetric (mean 68.5), which is why no log transform is used. Right: top Pearson correlations with the target; potential (in red) is the second strongest correlate, reactions leads, and ball_control ranks lower here than it later does on SHAP.

`potential`, a player's projected peak rating, correlates about 0.76 with `overall_rating`. A scouting platform would not have a clean, independently sourced version of this number at scoring time, so we drop it before training, the same logic that justifies dropping a row identifier. The work-rate columns hold invalid entries such as `norm`, `stoc` and stray digits; the underlying scale is ordinal (low, medium, high), so we map the valid labels to $\{0, 1, 2\}$ and treat every invalid entry as the modal value, medium. About 0.5% of rows have no target and are dropped outright, and for missing skill ratings we compute a column median on the cleaned table before the train/test split, a simplification we revisit as a limitation later, save the median to disk, and reuse it both at training and at serving time. Finally, we add a player's age (from the snapshot date and birthday), BMI, seven group averages over the FIFA skill categories (attacking, skill, movement, power, mentality, defending, goalkeeping), and a flag for goalkeepers.

Before any of this reaches the model, the `data_quality` pipeline checks the raw 6,000-row, 46-column sample against 19 rules covering value ranges, non-null targets and schema shape. All 19 pass with no warnings.

3.2 Modelling and validation

We compared four candidates with five-fold cross-validation on RMSE (Table 2). Both tree models clearly beat the linear baselines (RMSE 1.59 and 1.93, against 3.29 and 3.30): a rating isn't a simple weighted sum of the attributes, it depends on how they interact, and trees pick up those interactions while a linear model can't. HistGradientBoosting also beats Random Forest, and we selected it as

champion. On the held-out test set it reaches RMSE 1.45, R^2 0.954 and MAE 1.05.

Model	CV RMSE (pts)
HistGradientBoosting (champion)	**1.59**
Random Forest	1.93
Linear Regression	3.29
Ridge	3.30

Table 2: Five-fold cross-validated RMSE per candidate (`model_selection_report.json`).

An R^2 of 0.954 is high for a held-out test set, and there's a reason for that: FIFA computes `overall_rating` from roughly the same attributes we give the model, so a lot of what looks like prediction is the model recovering a known scoring rule. One caveat tempers that number: the split is a plain random one, and the table holds about 1.35 snapshots per player (4,418 unique players across 5,970 modelling rows), so the same player can end up in both train and test. To measure how much that flatters the result, we re-split by player instead of by row (`GroupShuffleSplit` on `player_api_id`, so no player appears on both sides). The cost is small: RMSE rises to 1.54, R^2 to 0.950, within-2 to 84.2 per cent, and every threshold from Table 1 still holds. We also re-checked the correlation table for any other column quietly duplicating the target the way `potential` does; nothing else came close.

Figure 3 backs that up with the actual diagnostics rather than asking the reader to take our word for it: residuals sit centred on zero with no funnel shape, and binned predictions track the observed rating closely through the dense middle of the range. The learning curve in the model-evaluation notebook covers only the 6,000-row sample, so it shows the sample is large enough for this result, not that the full dataset wouldn't push the numbers further.

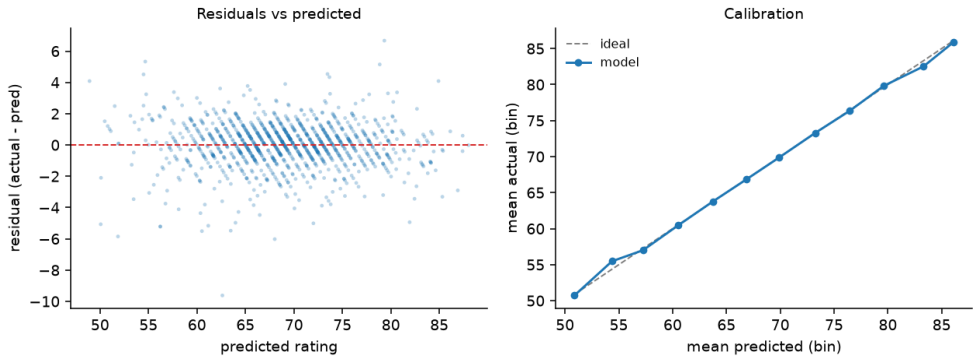
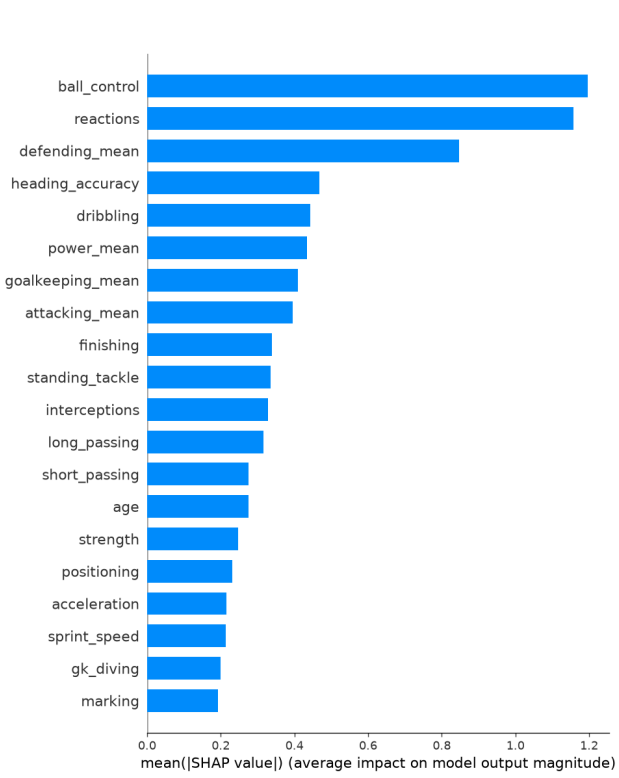


Figure 3: Left: residuals against predicted rating, no trend or widening spread. Right: binned predicted rating against the actual average in that bin, tracking the dashed identity line.

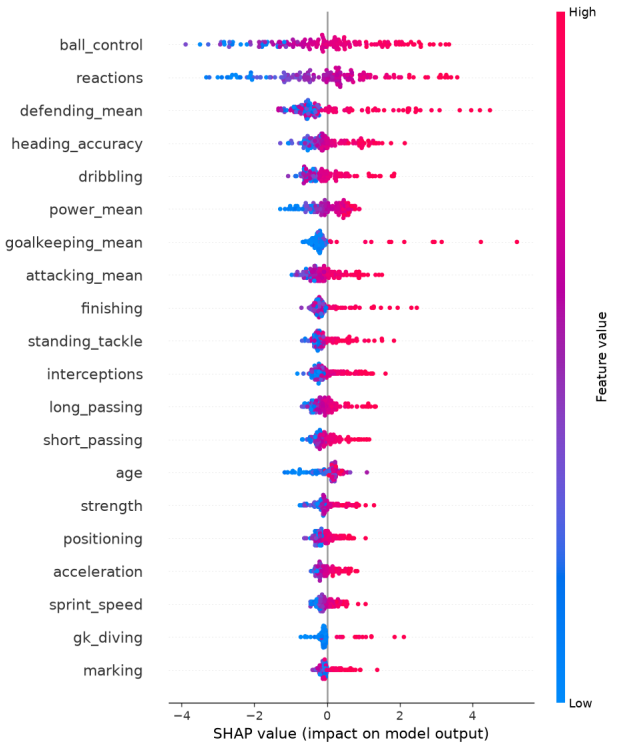
We also checked reproducibility by running the full DAG twice in a clean virtual environment: `pytest` passes, both runs land on the same champion, and the metrics in `data/08_reporting/model_metrics.json` match bit for bit (RMSE 1.4483) between runs.

3.3 Explainability (SHAP)

`model_train` computes SHAP values on the trained model and writes a beeswarm plot, a mean absolute-SHAP bar plot and `feature_importance.json`. If SHAP is unavailable in an environment, the pipeline falls back to permutation importance and records which method it used.



(a) Mean absolute SHAP value per feature.



(b) Beeswarm: per-sample SHAP impact, coloured by feature value.

Figure 4: Explainability for the champion model. ball_control and reactions dominate, followed by defending_mean, heading_accuracy and dribbling.

The ranking lines up with the EDA, where reactions alone correlates about 0.765 with the target, and ball_control is one of the strongest skill ratings we keep. Adding is_goalkeeper and goalkeeping_mean lets one model serve both keepers and outfield players; in the beeswarm, a high goalkeeping_mean sharply raises the prediction for that subset.

3.4 Drift monitoring

`data_drifts` compares a current batch against the training reference using Population Stability Index (PSI) and the Kolmogorov-Smirnov test per feature, declaring dataset drift once enough features cross a threshold. The committed drift_report.json is the clean run: 1 of 48 flagged, dataset_drift false. With 47 numeric KS tests at the 5% level we expect about two false positives by chance, so a single flag is consistent with no drift, and the one-third dataset-share threshold absorbs that noise without raising a dataset-level flag. To confirm the alarm fires when it should and not only stays quiet on clean data, we also ran it with drift.simulate_drift: true, which injects a broad shift into the same batch. That run flags 18 of 48 features and turns dataset_drift true, committed as drift_report_simulated.json; Figure 5 plots both runs side by side. Today the monitor watches input distributions only. Prediction drift, concept drift and an automatic retrain trigger are not built yet.

Area	Advantage now	Risk at scale	Mitigation
Compute (Pandas)	Simple, reproducible, no infra	Single machine; full table is about 184k rows	Port nodes to Spark or Dask, same node API (about 2 weeks)
Ingestion (CSV from SQLite)	One transparent script	Manual, point-in-time snapshot	Scheduled extract into a warehouse table (about 3 days)
Orchestration (Kedro CLI)	Modular, testable, layered catalog	No scheduling or retries	Deploy via `kedro-airflow` (about 1 week)
Tracking (MLflow, SQLite)	Free local versioning and registry	Not multi-user, no auth	Hosted MLflow server with object-store backend (about 3 days)
Feature store (parquet)	Lightweight offline store	No online serving, no point-in-time joins	Adopt Feast or Hopsworks (about 2 weeks)
Serving (FastAPI, Docker)	Standard, language-agnostic REST	Manual scaling	Kubernetes autoscaling, or MLflow serving (about 1 week)
Drift (PSI, KS, Evidently)	Interpretable, light dependencies	Batch only, heuristic thresholds	Scheduled job, alerting, retrain trigger (about 1 week)
Data quality (asserts)	Fast, no heavy dependency	Hand-maintained	Great Expectations suites in CI (about 3 days)

Table 3: Production roadmap: advantage, risk and mitigation per component.

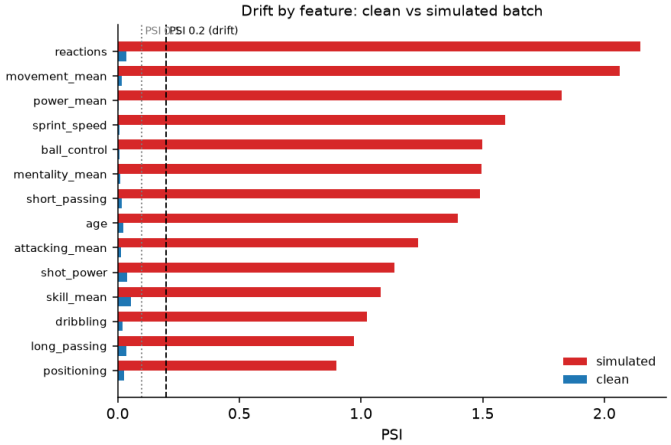


Figure 5: Per-feature PSI for the clean test batch (blue) and a simulated drifted batch (red). The clean run has no dataset-level drift flag; the simulated shift pushes enough targeted features past the 0.2 PSI threshold to cross the configured dataset-level rule.

4 From proof of concept to production

For each technology choice, Table 3 lists what it buys us now, where it breaks at scale, and the next step with a rough effort estimate.

A feature store with CI checks is the right place to enforce rules like the `potential` drop, and to add it back automatically if a clean, independently sourced version ever became available. Closing the drift-monitoring gaps from Section 3.4 is the main maturity step from here. Reproducibility rests on a single seed (`parameters.yml`), pinned dependencies and persisted Kedro layers, which we verified end to end in Section 3.2.

5 Packages and versions

All versions are frozen in `requirements-lock.txt` (around 250 pinned packages). Table 4 lists the core stack and what each piece is doing in the pipeline.

Package	Version	Role
kedro / kedro-datasets	0.19.x / 4.x	Pipeline orchestration and the layered data catalog
pandas / numpy	2.x / 2.x	Data handling and numerics
pyarrow	15+	Parquet I/O for the catalog
scikit-learn	1.x	Preprocessing, the model zoo, metrics
scipy	1.1x	KS test for drift
mlflow	3.x	Experiment tracking and the model registry
shap / matplotlib	0.5x / 3.x	Explainability and plots
evidently	0.7.x	HTML drift report
fastapi / uvicorn / pydantic	see lock file	Model serving
pytest / pytest-cov	8.x / 5.x	Test suite (25 tests)

Table 4: Core packages and their role in the pipeline.